Terence: Design and Implementation of an Angry Birds Playing Robot

Authors: Mohammad Arbab, Camilo Garrido Fernandez, Evelyn Huerta

March 2026

1 Task and Basic Concept

The objective of our robot was to play a real life version of the Angry Birds video game, in which it uses a slingshot to launch racquetballs (representing the birds) at the targets (3D printed pigs). The robot is able to detect and pick up blue colored balls (the birds / ammo), load it into the slingshot, detect and aim at the targets, and launch the balls to knock them down.

The robot should expect that a human might...

- knock down targets themselves (using their own slingshot)
- put up new targets
- add balls to, or take balls from, the robot's ball pen
- load a ball into, or remove a ball from, the robot's slingshot
- bump into the robot

and should be able to react and continue operating through all of these interactions.

The unique assembly of the robot with the slingshot mounted on the back of the robot enabled it to operate effectively in two directions. In the forward direction the robot was able to collect and load balls, while in the reverse direction the robot aimed and launched balls at the targets. However, this flexibility lead to many challenges when solving for the correct kinematic solution.

2 Mechanism and Hardware

The robot is comprised of two joint mechanical systems: a 4-DOF robot arm and a slingshot. By aligning the slingshot symmetrically to the gripper, as shown in Figure 2a, and mounting it to the bracket above the base motor, the robot's interaction with the slingshot is simplified to planar motion along the vertical plane oriented to the angle of the base motor—compared to alternative solutions such as a separate table-mounted slingshot. This ensures that the 3-DOFs needed to pick up balls in (x,y,z) also suffice for manipulating the slingshot. A fourth DOF is added to control the grippers vertical angle from the table, ϕ, which facilitates tasks where certain gripper orientations are preferable—such as picking up balls with the gripper pointed down or grabbing the slingshot handle with the gripper perpendicular to it.

Because the balls can be placed fairly close to the robot, and the dimensions of the slingshot can be fairly easily scaled to be within the robot's reach, the length of the arm links were simply chosen to be 0.224 m. This results in the robot arm having a roughly 0.5 m reach about the central axis of the base motor.

To aid with motor selection, we estimated the force required to launch the racquetballs (which weigh about 0.040 kg) a distance of 3 meters (about as far as target can be from the robot) with 4 rubber bands whose combined spring constant measured about 50 N/m, as follows:

$$m = 0.040\,\text{kg}, \quad g = 9.81\,\text{m/s}, \quad d = 3.0\,\text{m}, \quad k = 50\,\text{N/m} \tag{1}$$

$$v_0 = \sqrt{g \cdot d} \approx 5.42\,\text{m/s} \tag{2}$$

$$F = v_0\sqrt{k \cdot m} \approx 7.67\,\text{N} \tag{3}$$

This means that the slingshot would have to be pulled back by a distance of approximately:

$$\Delta x = \frac{F}{k} \approx 0.15\,\text{m} \tag{4}$$

The lever arm of this force on the robot motors would realistically never be more than the length of one arm link when in a launching position (arms bent elbow-up). Thus, an upper-bound on the torque at the motors while pulling would be:

$$\tau = F \cdot l \approx 7.67\,\text{N} \cdot 0.224\,\text{m} \approx 1.72\,\text{Nm} \tag{5}$$

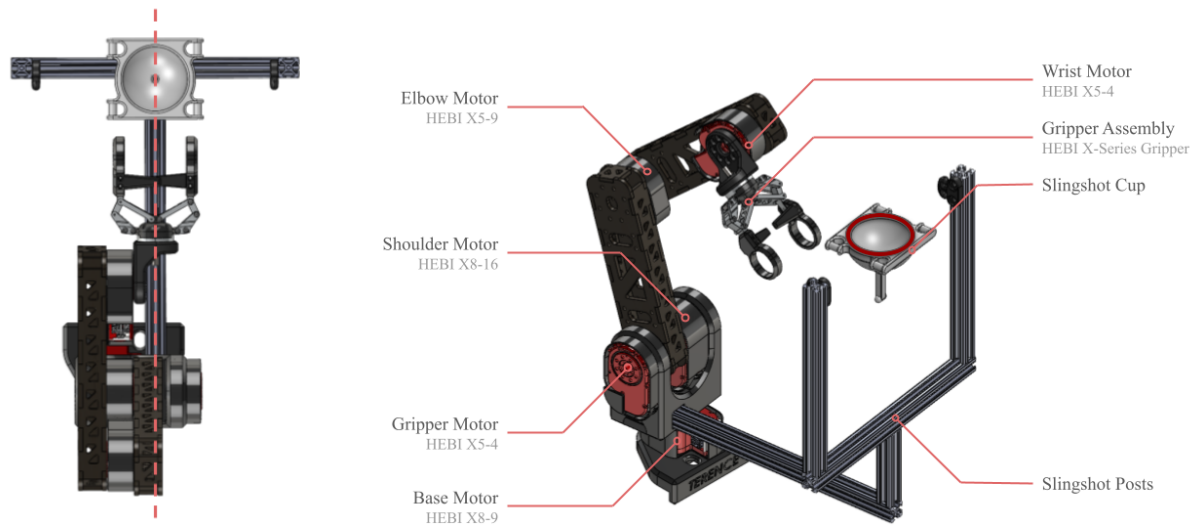
Thus, virtually any HEBI X-Series motors except the X5-1 should have enough torque to pull back the slingshot as needed (see Figure 1).

X-SERIES ACTUATOR — TECHNICAL SPECIFICATIONS

CONFIGURATION	X5-1	X5-4	X5-9	X8-3	X8-9	X8-16
Peak Torque	2.5 N-m	7 N-m	13 N-m	7 N-m	20 N-m	38 N-m
Cont. Torque	1.3 N-m	4 N-m	8 N-m	3 N-m	8 N-m	16 N-m
Max Speed	90 RPM	32 RPM	14 RPM	84 RPM	30 RPM	15 RPM
Mass	330g	345g	360g	455g	475g	490g
Dimensions	110mm x 73mm x 31mm 15mm hollow bore			110mm x 73mm x 45mm 15mm hollow bore		

Figure 1: HEBI X-Series Motor Specs

However, since the shoulder would hold up the majority of the robot arm's weight, in addition to being the most involved in pulling back the slingshot cup, and is at the base of the robot arm (so its own mass is relatively low-cost) an X8-16 was selected to offer a comfortable margin of safety. Similarly, the elbow motor contributes the second most in bearing these loads, but its weight does need to be carried by the shoulder



(a) Horizontal alignment of the gripper and slingshot.

(b) Overview of key mechanical components.

Figure 2: Mechanical overview of the robot.

motor, so the lighter X5-9 was selected for it. Additionally, an X8-9 motor was selected for the base motor to balance high torque for moving the eccentric masses of the robot arm and slingshot, without significantly compromising on speed (compared to an X5-4) so that the robot could quickly turn 180° to switch from grabbing a ball to aiming and launching it. The wrist and gripper motors were left as X5-4's.

The slingshot protrudes out about 0.336 m in front of the robot arm and 0.361 m above the base motor—allowing plenty of room for pulling the slingshot cup back 0.15 m as predicted in equation (4). The slingshot cup has a diameter of 8.00 cm, leaving a 1.15 cm radial margin of error when loading the 5.70 cm diameter racquetballs. Rubber bands are used to connect the slingshot cup to hooks (for easy removal/replacement) on the slingshot posts. A handle is screwed onto the slingshot cup. Gripper attachments were designed with slots that mechanically center the hemispherical ridges on the handle of the slingshot cup when the the cup is pulled back (Figure 3). Additionally, angled fingers on the gripper attachment help push the handle forward into place if/when the slingshot cup tilts.

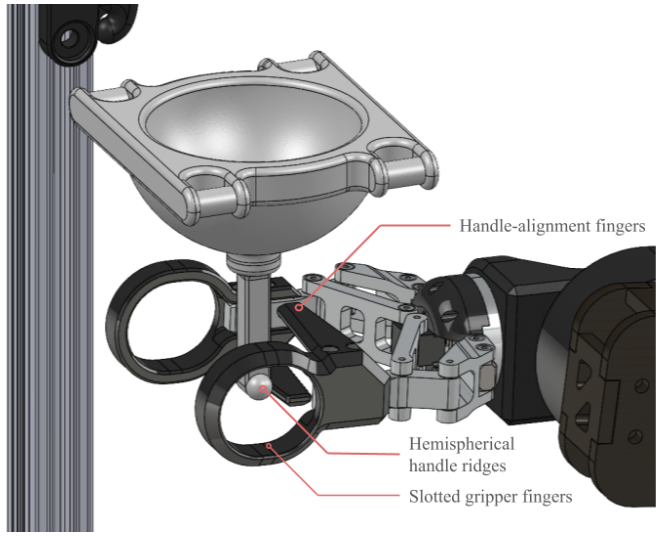


Figure 3: Gripper and sligshot-cup mechanical alignment features.

3 Task Space, Kinematics, Trajectories, Precision, Continuity (Basic Robotics)

The robot operates in a (x, y, z, ϕ) task space, where ϕ is the wrist pitch angle used when interacting with the slingshot handle. The inverse kinematics is split into two solvers depending on context: a full 4-DOF damped Newton–Raphson solver for the arm, and a closed-form 1-DOF solver for aligning the slingshot base motor.

3.1 Inverse Kinematics

The arm inverse kinematics uses a damped pseudo-inverse ($\lambda = 0.5$) to handle singularities, running for up to $N = 300$ iterations until achieving millimeter-accuracy (i.e., the position error falls below $\varepsilon = 10^{-3}$ m). Solution multiplicity is managed via direction-seeded initialization: a "forward" seed places the arm over the ball pen, while a "backward" seed orients it toward the slingshot. When a task-space orientation ϕ is specified, a secondary wrist task is projected into the null space of the primary position task so it never corrupts end-effector position. Converged joint angles are wrapped to their respective motor limits before use.

The slingshot base inverse kinematics is solved in closed form: the lateral offset d between the base motor and the slingshot extrusion introduces a fixed angular correction ψ that is computed analytically from the goal direction, requiring no iteration.

The algorithms are written in more detail in Appendix A (Algorithms 6 and 7).

3.2 Gravity Model

The robot uses a gravity compensation model (Algorithm 1) with five manually-tuned coefficients A–E. Torques are computed outward-to-inward (wrist $\rightarrow$ shoulder) and blended in linearly over the first 5 seconds of operation to avoid startup discontinuities. This model allows the robot to hold its configuration and resist gravity mid-spline, which is critical for safe collision recovery.

Algorithm 1 Joint Gravity Torque Model with Startup Blend

Require: Joint positions $\mathbf{q} \in \mathbb{R}^5$, elapsed time t, learned parameters A, B, C, D, E
Ensure: Gravity compensation torque vector $\boldsymbol{\tau}_{\text{grav}} \in \mathbb{R}^5$

Compute torques from the distal joint outward:

$$\begin{aligned}
\tau_{\text{wrist}} &= E \sin(-q_2 + q_3 - q_4) \\
\tau_{\text{elbow}} &= -\tau_{\text{wrist}} + C \sin(-q_2 + q_3) + D \cos(-q_2 + q_3) \\
\tau_{\text{shoulder}} &= -\tau_{\text{elbow}} + A \sin(-q_2) + B \cos^3(-q_2)
\end{aligned}$$

$\boldsymbol{\tau}_{\text{grav}} = \begin{bmatrix} 0, \ 0, \ \tau_{\text{shoulder}}, \ \tau_{\text{elbow}}, \ \tau_{\text{wrist}} \end{bmatrix}^\top$

return $\alpha(t)\, \boldsymbol{\tau}_{\text{grav}}$, where $\alpha(t) = \min\left(\frac{t}{5}, \ 1\right)$

Note: although the cosine terms should not be necessary in theory—because the robot arm's zero position points straight up—we found that the robot would be stiffer on one side without them.

3.3 Motion Detectors: Collision and Gripper

Two special-purpose detectors run continuously. Collision detection (Algorithm 2) compares commanded and actual joint state across position, velocity, and effort, excluding the gripper joint which has its own detector. Because the slingshot pull intentionally generates large effort deviations, all three thresholds are doubled during that phase to suppress false positives. When a collision is detected, gravity compensation holds the arm in place until contact is cleared.

Algorithm 2 Collision Detection via Command–Actual Error Thresholding

Require: Commanded state $(\mathbf{q}^d, \dot{\mathbf{q}}^d, \boldsymbol{\tau}^d)$, actual state $(\mathbf{q}^a, \dot{\mathbf{q}}^a, \boldsymbol{\tau}^a)$, thresholds $(\bar{e}_q, \bar{e}_{\dot{q}}, \bar{e}_\tau)$, gripper index g, abort flag `abort`
Ensure: Updated `abort` flag; collision status published

if `abort` **or** any state vector is `None` **then return** ▷ Skip if already aborted or uninitialised
end if
Define $(\cdot)_{\setminus g}$ as the vector with index g removed
$e_q = \|\mathbf{q}^d_{\setminus g} - \mathbf{q}^a_{\setminus g}\|$
$e_{\dot{q}} = \|\dot{\mathbf{q}}^d_{\setminus g} - \dot{\mathbf{q}}^a_{\setminus g}\|$
$e_\tau = \|\boldsymbol{\tau}^d_{\setminus g} - \boldsymbol{\tau}^a_{\setminus g}\|$
$(\bar{e}_q, \bar{e}_{\dot{q}}, \bar{e}_\tau) = 2 \times (\bar{e}_q, \bar{e}_{\dot{q}}, \bar{e}_\tau)$ ▷ Loosen thresholds during slingshot pull
if $e_q > \bar{e}_q$ **then**
    `abort` = `True`; log position collision
else if $e_{\dot{q}} > \bar{e}_{\dot{q}}$ **then**
    `abort` = `True`; log velocity collision
else if $e_\tau > \bar{e}_\tau$ **then**
    `abort` = `True`; log effort collision
end if
Publish `abort` as collision status message

The gripper detector (Algorithm 3) identifies a failed grasp by checking that the gripper reached its commanded closed position with neither motion nor resistive effort — indicating it closed on air. This triggers an immediate abort and recovery, which is essential since all subsequent modes assume a ball is in hand.

Algorithm 3 Gripper Empty-Close Detection

Require: Gripper actual position q_g^a, commanded position q_g^d, actual velocity $\dot{q}_g^a$, actual effort τ_g^a
Ensure: Boolean `fail_close`; status published

1: `fail_close` = `False`
2: **if** all four conditions hold: **then**

$$q_g^d < -0.1 \qquad \text{(close command issued)}$$
$$|q_g^d - q_g^a| < 0.05 \qquad \text{(reached commanded position)}$$
$$|\dot{q}_g^a| < 0.1 \qquad \text{(gripper not moving)}$$
$$|\tau_g^a| < 0.3 \qquad \text{(no resistive effort)}$$

3: `fail_close` = `True` ▷ Closed without contacting an object
4: **end if**
5: Publish `fail_close` as gripper status message

4 Cameras and Visual Detectors

Our system uses a standard RGB camera and a depth camera. The standard camera is mounted above the robot workspace which includes the ball pen and is responsible for detecting the position of the balls. It is also responsible for detecting when a ball is loaded into the slingshot cup. This overhead camera performs a one-time calibration using four AruCo markers at known world positions. After this initial calibration, it uses logic to estimate the position of the fourth marker in cases where only three are visible. At the same time, the ball mapping node uses HSV color thresholding to identify the balls, creating circular contours and determining their centroid in pixel coordinates. The AruCo markers and the HSV tuning are integrated together in a mapping from pixel to world coordinates. All balls detected are appended to a BallArray sent as a message to the brain node.

There were some issues encountered during ball detection, specifically when balls would bunch together and be labeled as one large ball. This made determining whether or not a ball is loaded in the slingshot a tricky task, since the camera does not have depth sensing. We were originally deciding if a ball was loaded or not through the area of the largest contour; a loaded ball had an almost twice as large area as any single ball in the ball pen, since it was located closer to the mounted camera in the z-direction. However, this logic is faulty once you consider how a close group of balls may be interpreted as one large ball which triggers a false positive for a loaded ball. Thus, a red perimeter was added to the top of the slingshot cup to detect if a ball is located within the red ring.

The main task of the depth camera is to detect the position of the targets. There are not any AruCo markers for the real sense camera, but we leverage the real world coordinates of the camera and use the depth information obtained by the data to convert pixel coordinates to world coordinates. The green pig targets are tuned with HSV and then appended to a TargetArray, which is also sent as a message to the brain node.

Both of the cameras send their respective messages every 20-30 ms. The information received from these cameras coupled with the logic for the object detection leads to almost perfect accuracy when picking up balls, determining if a ball is loaded, and hitting targets. The set up of these cameras are able to handle all of the situations tested so far.

5 ROS Software Architecture

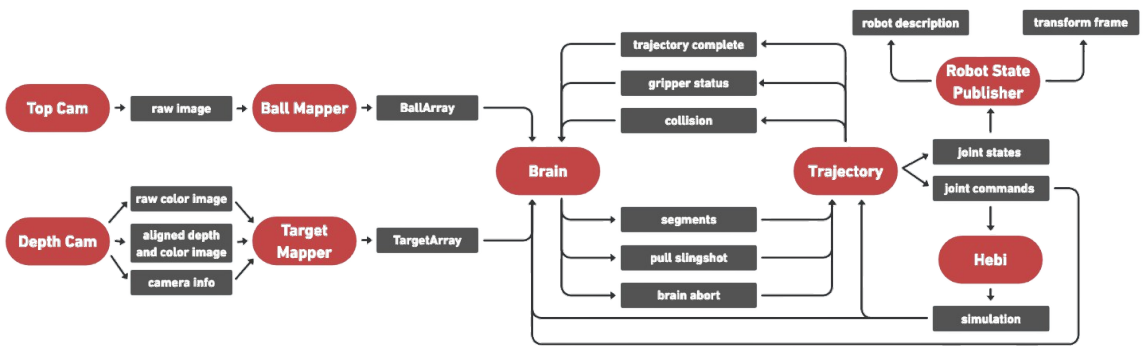


Figure 4: ROS Architecture

The system is composed of eight ROS nodes that work together to perceive the environment (perception), make decisions (control), and move the robot (execution). The diagram above shows how information flows between these nodes and the sections below describe each node in more depth.

Top Cam

This node is a simple USB camera driver. It captures images from the overhead camera and sends out the raw image, so other parts of the system can use it.

Ball Mapper

This node processes the image from the top camera, detects balls based on their HSV thresholds, and calculates their positions on the table. Then it sends a list of all ball positions to the Brain.

Depth Cam

This node is an Intel RealSense D435 depth camera driver. It provides three types of data: a normal color image, a depth image (with depth information), and camera calibration information.

Target Mapper

This node uses the depth camera data to locate targets in 3D space based on their HSV limits. It combines color and depth information to figure out where targets are, then sends those positions to the Brain.

Brain

The Brain handles all the modal logic and is essentially the decision maker of the system. It receives the positions of balls and targets, along with feedback from the robot about it's current position and status, such as whether a completed a motion or if there was a collision. Given all this information, the Brain decides the next actions of the robot and sends movement instructions, segments, to the Trajectory node.

Trajectory

The Trajectory node receives segments from the Brain and smoothly splines between them. It calculates how the robot's joints should move over time, handles things like collision checking and gripper control, and then sends motor commands to the hardware. It also sends feedback back to the Brain, such as when a motion is finished or if something went wrong.

Hebi

The Hebi node directly controls the robot's motors. It receives commands from the Trajectory node, moves the physical robot, and reports back the current joint positions and velocities.

Robot State Publisher

The Robot State Publisher node keeps track of the robot's structure and how all its parts are positioned. It takes in the current joint states and publishes the full robot description, including how all parts are connected and positioned in space. This information is used by other nodes to understand where the robot is and how it is moving.

6 Understanding the World

6.1 Ball Reachability Detection

If the robot tries to grab a ball that is rolling around in its ball-pen, it will likely miss because by the time that the robot reaches the ball's initial position the ball will already be somewhere else. To avoid this, the robot only reaches for balls that are "stable" (i.e, the detected XY position of the ball has not changed appreciably between two consecutive camera frames). Let $\mathbf{b}_k = [x, y]^\top$ be the current ball position and $\mathbf{b}_{k-1}^{(i)}$ the i-th ball in the previous frame.

Algorithm 4 Ball Stability Check

Require: Current ball $\mathbf{b}_k \in \mathbb{R}^2$, previous ball array $\{\mathbf{b}_{k-1}^{(i)}\}$, stability threshold $\delta_s = 0.01\,\mathrm{m}$
Ensure: Boolean `stable`

```
stable = False
for each b_{k-1}^{(i)} in previous frame do
    if ||b_k − b_{k-1}^{(i)}|| < δ_s then
        stable = True; break
    end if
end forreturn stable
```

Even if a ball is stable, it is only added to the trajectory queue if it lies within the [X,Y] coordinate bounds of the ball-pen region and is reachable (i.e., the Newton Raphson for the inverse kinematics to reach the ball position converges to a solution). This ensures that the robot does not try to reach for a ball that a human might be holding outside of the ball-pen.

6.2 Target Loss Detection During Launch

While the robot is executing a launch trajectory, it continuously monitors whether the selected target is still there. This is effectively the same as the Ball Stability Check 4 just with target positions and a slightly larger error threshold (0.030 m) since the targets are larger than balls.

6.3 Slingshot Cup Loading Detection and Recovery

The vision system continuously publishes the XY centroid of the slingshot cup. A ball is declared *loaded* if any detected ball lies within d of the cup centroid (where d is the actual diameter of a racquetball) in any direction. If a ball is unexpectedly detected as loaded before it should be (i.e., before the GrabHandle mode), the brain triggers an appropriate response.

Algorithm 5 Cup Loading Detection and Early-Load Recovery

Require: Cup centroid $\mathbf{c} \in \mathbb{R}^2$, detected balls $\{\mathbf{b}_i\}$, ball diameter d, current mode, gripper command q_g^d, end-effector position $\mathbf{p}_{\text{tip}}$, cup position $\mathbf{p}_{\text{cup}}$

Ensure: `ball_loaded` flag; mode transition if early load detected

```
ball_loaded = False
for each ball b_i do
    if ||b_i^{xy} − c|| ≤ d then
        ball_loaded = True; break
    end if
end for
if ball_loaded and mode ≤ GrabHandle then
    Check whether the gripper is near the cup:
```

$$\texttt{near_cup} \;=\; \big(\|\mathbf{p}_{\text{tip}} - \mathbf{p}_{\text{cup}}\| \le 2\,d\big)$$

```
    if not near_cup then                                        ▷ Unexpected early load
        Clear trajectory queue; send abort
        if gripper is closed (q_g^d < −0.1) and no prior missed close then
            Plan deposit: move tip to random pen position, open gripper   ▷ Return held ball to pen
        end if
        Clear abort; skip directly to mode GrabHandle   ▷ Prepare to launch the loaded ball
    end if
end if
```

The `near_cup` guard prevents false recoveries when the gripper is legitimately positioned over the cup when loading the ball. The random deposit target is sampled uniformly from $x \in [0.30, 0.45]\,\text{m}$, $y \in [0.20, 0.35]\,\text{m}$ (near the center of the ball pen), at height $z = 3r_b$ (to clear the ball pen rails).

7 Actions: Normal Behaviors, Failure Recoveries, and other Special Features

The system is quite sophisticated and has various different failure recoveries and contingencies to ensure a continuous task flow. The nominal task flow is described and depicted below:

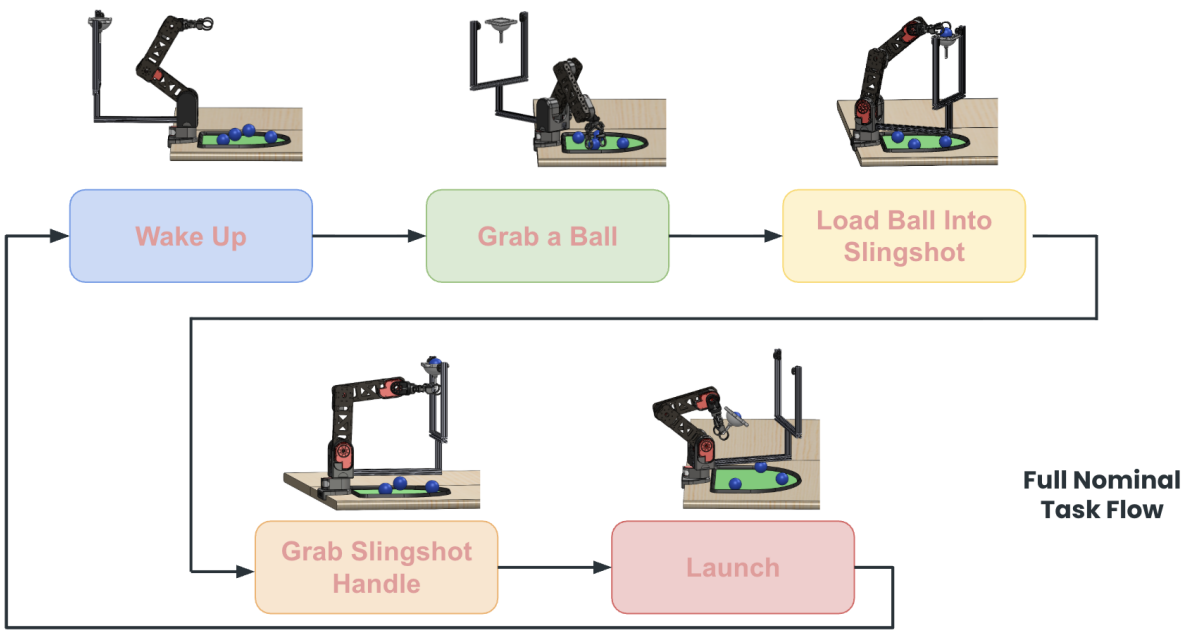


Figure 5: Full Nominal Task Flow

1. Wake Up - The robot moves to a ready position after start up.
2. Choose Ball - The robot detects a ball, move the open gripper to the ball, closes the gripper, and lifts the ball slightly above the ball pen.
3. Load ball - The robot arm loads the ball into the slingshot cup behind it and simultaneously rotates the base 180° so that the slingshot faces the target zone.
4. Grab handle - The robot arm grabs the slingshot handle.
5. Launch - The base is rotated towards the desired target and the arm pulls back, then the gripper releases the handle to launch the ball.

As mentioned before, the system has several recovery modes, so that the robot responds to any scenario in a logical matter. In order to achieve this, we have a couple of continual checks that provide useful information about the robots current status. The robot continually monitors collision status, gripper status, and locations of balls and targets. From this information, we are able to handle the following failures or unexpected actions:

- **Collision Detected:**
  If a collision is detected anywhere in the workflow, then the robot stops for a second and attempts to continue on its path. To be clear, if there is continuous pressure applied to the robot a collision will also continue to be detected, so the robot will not move.
- **Ball Detected in Slingshot:**
  If a ball is detected in the slingshot during almost any point in the task flow, except for Launch and after the gripper is a specific height in LoadBall, the robot will skip to GrabHandle. This prevents the robot from loading a ball in the cup if one is already loaded. This also enables it to take advantage of the loaded ball, so that it does not waste time by performing unnecessary tasks.
  Note: The ball detection logic is not applied in Launch because there are times that the angle of the slingshot cup prevents the camera from detecting a ball. It is also not applied after the gripper is a specific height in LoadBall, because it could trigger a false positive. The robot cannot differentiate between the gripper holding the ball directly above the slingshot cup and an actual loaded ball.

- **Recovery Modes from WakeUp to ChooseBall:**
  - If there aren't any balls detected in the ball pen, then the robot rests in its current position until a ball is detected.

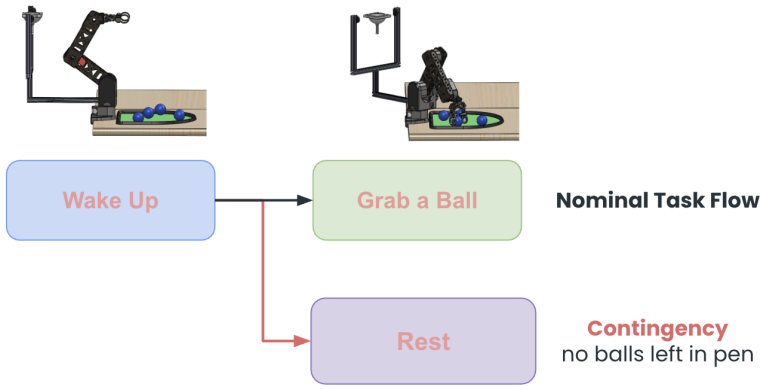


Figure 6: Enter Caption

- **Recovery Modes in ChooseBall and LoadBall:**
  - If a ball is not successfully grasped during ChooseBall, then it will constantly attempt to grab a ball. This usually occurs when multiple balls are grouped together and interpreted as one large ball by our detection. The robot arm will continuously attempt to grab the "large" ball, consequently breaking the group of balls apart until one is successfully identified and picked up.
  - If a ball slips out of the gripper during LoadBall, then the robot returns to ChooseBall and selects another ball.

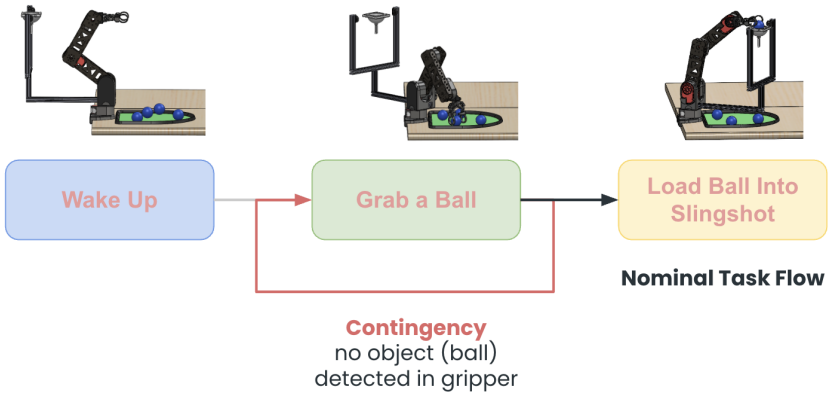


Figure 7: Enter Caption

- **Recovery Modes in GrabHandle:**
  - If a ball is not detected in the slingshot cup, then the robot returns to WakeUp to restart the task flow logic.
  - If the slingshot handle is not detected in the gripper, then the robot attempts to grip the handle again while the ball remains loaded.
    Note: The detection of the of the slingshot handle in the grippers was not very accurate, but

the logic was in place. Since the detection was not accurate, we biased towards false positives, i.e. handle between grippers, rather than false negatives, i.e. handle not between grippers. This is because the constant detection for ball in the slingshot cup handled the false negatives.

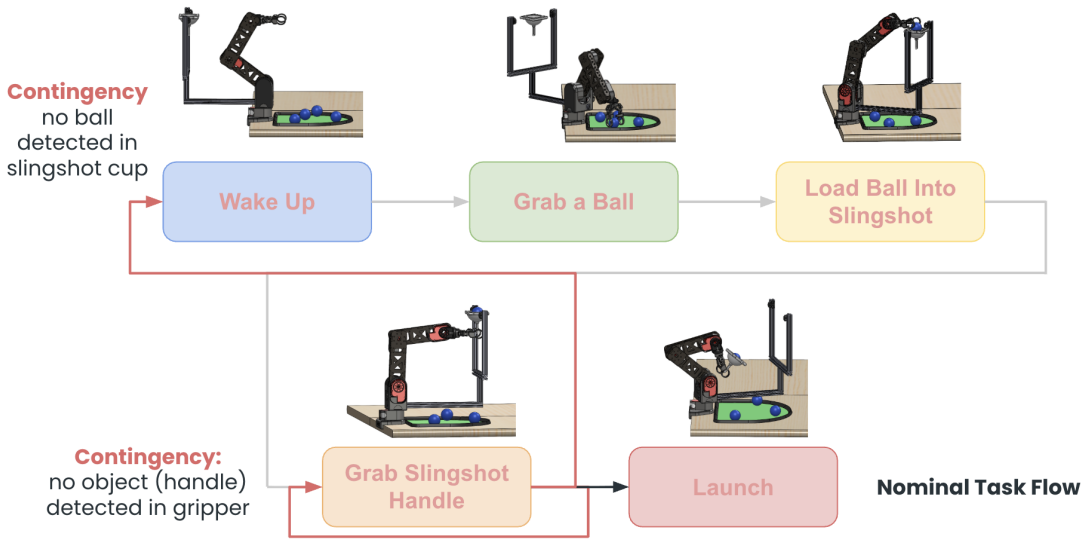


Figure 8: Enter Caption

- **Recovery Modes in Launch:**
  - If there are not any targets detected, then the robot rests in its current position until a target is detected.
  - If the target the robot was previously aiming for was knocked down, then the robot aims for another target.

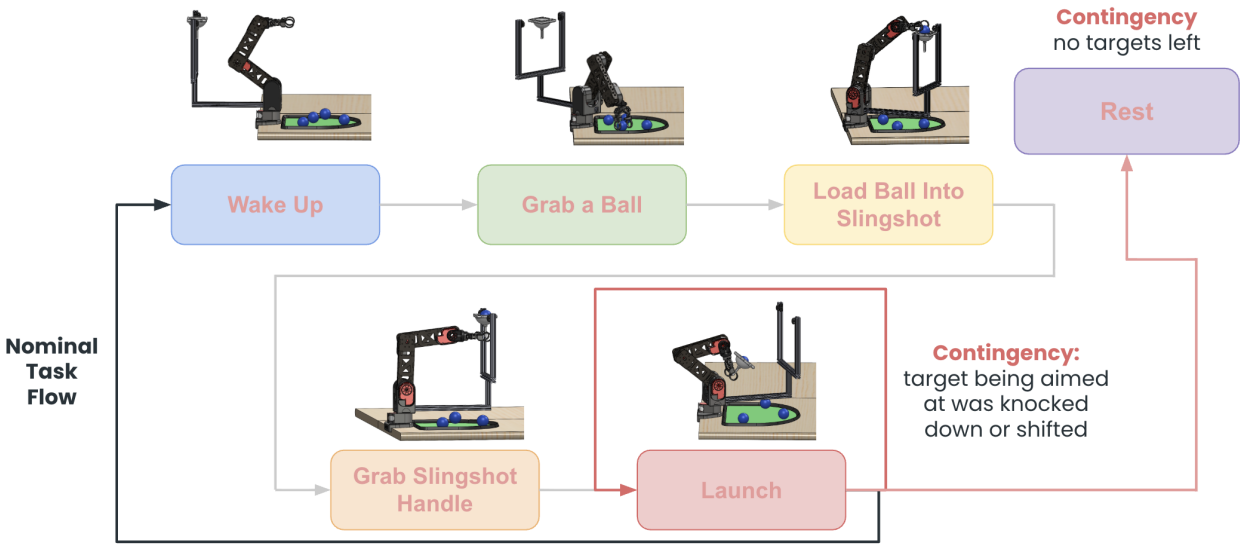


Figure 9: Enter Caption

8 Shortcomings, Outstanding Items, and Lessons Learned

Despite continuous iteration and refinement, there were still some limitations in the final robot. One of the most prominent challenges was the reliability of grabbing the slingshot handle. Due to the slimness of the slingshot handle, the gripper motor would sense little positional and torque difference between when the gripper ends successfully clasped the handle versus when they closed on themselves. As a result, the robot struggled to detect when it failed to close on the handle. When the slingshot cup wound up significantly, the robot would sometimes get caught in a loop of trying to grab the handle, missing and going through the launch motions, and then trying to grab the handle again because the ball would still be loaded in the slingshot cup. We had hoped to solve the gripper detection issue mechanically by making a wider slingshot handle, but ran out of time to do so. This would have enabled more intelligent failure-handling—such as varying the waypoint for grabbing the handle when consecutive misses are detected. There were also some issues related to ball collection related to gripper feedback. Detecting when a ball was successfully picked up required careful tuning of gripper parameters and thresholds. There were delayed responses after unsuccessful pickups that could be improved through finetuning the gripper effort, position, and velocity thresholds. These inconsistencies were also likely caused by hardware constraints. In particular, the gripper cable would occasionally bend and become lodged between the arm links of the robot, introducing variability in performance.

Reliable detection of a ball loaded in the slingshot cup was a critical component of the system, and several approaches were explored before arriving at a workable solution. In retrospect, incorporating a depth camera would have significantly simplified this process. The 3D ball positions could have been compared to the slingshot cup position (determined through the forward kinematics and calibrated using the ArUco markers on the table) for more robust and direct detection.

Additionally, we attempted to implement a routine that would auto-tune the spring constants of the slingshot's rubber bands (Algorithm 8 in Appendix C), however it virtually never worked. We suspect the issue was that the gravity compensation model is an approximation, and any residual error in the predicted gravity torques would directly contaminate the measured torques that the system believes are not due to gravity (and therefore the extrapolated force believed to be acting on the end-effector). Additional unmodeled forces such as motor friction and joint-level mechanical compliance likely further corrupt the effort readings, causing the estimated spring constant to vary significantly across pulls. A more robust alternative might have been a simpler calibration routine in which the robot periodically pulls the slingshot cup by a fixed displacement using a single motor at a fixed joint configuration. However, this was not implemented due to time constraints.

Finally, the transitions between waypoints did not always appear very fluid (despite technically being smooth) and could have been improved by spending more time tuning the desired final velocities.

Appendix

A Inverse Kinematics

A.1 Robot Arm Inverse Kinematics

Algorithm 6 Robot Arm IK via Damped Newton–Raphson with Null-Space Orientation Control

Require: Goal pose $x_{\text{goal}} \in \mathbb{R}^3$ (position) or $\mathbb{R}^4$ (position + wrist angle φ_{goal}), direction $\in \{\text{f}, \text{b}\}$, damping λ, precision ε, max iterations N

Ensure: Joint configuration $q \in \mathbb{R}^4$, or `None` on failure

Initialize $q = q_{\text{seed}}$ **if** seed provided, **else** direction-dependent default
for $i = 1, \ldots, N$ **do**
  $(p_{\text{tip}},\ J_v) = \textsc{ForwardKin}(q)$ ▷ tip position and 3×4 translational Jacobian
  $\Delta x = x_{\text{goal}}[:3] - p_{\text{tip}}$ ▷ Cartesian position error
  $\tilde{J}^+ = J_v^\top \left(J_v J_v^\top + \lambda^2 I_3\right)^{-1}$ ▷ Damped pseudo-inverse (Levenberg–Marquardt)
  $\Delta q = \tilde{J}^+ \Delta x$ ▷ Primary task step
  if $|\, x_{\text{goal}}| = 4$ **then** ▷ Orientation secondary task
    Compute current wrist angle:

$$\varphi_{\text{curr}} = \begin{cases} q_1 - q_2 + q_3 & \text{(forward)} \\ -q_1 + q_2 - q_3 & \text{(backward)} \end{cases}$$

    $\delta\varphi = \left(\varphi_{\text{goal}} - \varphi_{\text{curr}} + \pi\right) \bmod 2\pi \ - \ \pi$ ▷ Wrap error to $(-\pi, \pi]$
    $J_\omega = [0,\ 1,\ -1,\ 1]$ (forward) or $[0,\ -1,\ 1,\ -1]$ (backward)
    $\Delta q \mathrel{+}= \left(I - J_v^+ J_v\right)\ k_s\ J_\omega^\top\ \delta\varphi$ ▷ Null-space projection, gain k_s
  end if
  $q = q + \Delta q$
  if $\|\Delta x\| < \varepsilon$ **then return** q (wrapped within desired motor ranges)
  end if
end for
  return `None` ▷ Convergence failed

A.2 Slingshot Inverse Kinematics

Algorithm 7 Slingshot Base-Angle IK (Closed Form)

Require: Goal position $x_{\text{goal}} \in \mathbb{R}^2$ (XY plane), lateral offset d from base motor to slingshot extrusion

Ensure: Base joint angle $q_{\text{base}} \in \mathbb{R}$

$r_g = \sqrt{x_g^2 + y_g^2}$ ▷ Radial distance to goal
$\alpha = \arctan 2(y_g,\ x_g)$ ▷ Azimuth angle to goal
$\psi = \arctan 2\left(\dfrac{d}{r_g},\ \sqrt{1 - \dfrac{d^2}{r_g^2}}\right)$ ▷ Angular offset due to lateral extrusion
return $q_{\text{base}} = \alpha + \psi + \dfrac{\pi}{2}$

B Slingshot Kinematics and Trajectory Optimization

B.1 Coordinate Frames and Launch Geometry

Let $\mathbf{p}_{\text{eq}} \in \mathbb{R}^3$ be the slingshot cup equilibrium position, d the pull distance, ℓ_h the handle length, θ the launch elevation angle, and β the base (azimuth) angle. The unit launch direction vector is

$$\hat{n} = \begin{bmatrix} \cos\theta \sin\beta \\ -\cos\theta\cos\beta \\ \sin\theta \end{bmatrix}.$$

Pulling back by d from the equilibrium point (against $\hat{n}$) sets the launch position:

$$\mathbf{p}_0 = \mathbf{p}_{\text{eq}} - (d + \ell_h)\,\hat{n}.$$

B.2 Spring Launch Velocity

Assuming full spring energy conversion to kinetic energy ($\frac{1}{2}kd^2 = \frac{1}{2}mv_0^2$):

$$v_0 = d\sqrt{\frac{k}{m}}.$$

The initial velocity vector is $\mathbf{v}_0 = v_0\,\hat{n}$.

B.3 Ballistic Trajectory

Under gravity g (drag neglected), the projectile position at time t is

$$\mathbf{r}(t) = \mathbf{p}_0 + \mathbf{v}_0\,t - \tfrac{1}{2}g\,t^2\,\hat{e}_z,$$

giving final position $\mathbf{p}_f = \mathbf{r}(t_f)$ and final vertical velocity $v_{f,z} = v_0 \sin\theta - g\,t_f$.

B.4 Optimization Problem

The parameters $\boldsymbol{\xi} = [d,\ t_f,\ \theta]^\top$ are chosen to minimize the cost

$$\mathcal{L}(\boldsymbol{\xi}) = \underbrace{\omega_1\,\|\mathbf{p}_f - \mathbf{p}_{\text{target}}\|^2}_{\text{accuracy}} + \underbrace{\omega_2\,d^2 + \omega_3\,v_{f,z}^2}_{\text{efficiency}} + \underbrace{\omega_4\,\max\!\big(0,\ \theta_{\min} - \theta\big)^2}_{\text{min. angle}} + \underbrace{\omega_5\,\max\!\big(0,\ v_{\min} - \|\mathbf{v}_f\|\big)^2}_{\text{min. speed}}$$

subject to box constraints $d \in [0.05, 0.30]\,\text{m}$, $t_f \in [0.1, 5.0]\,\text{s}$, $\theta \in [45°, 60°]$.

Solver. $\mathcal{L}$ is non-convex (the accuracy term couples all three parameters nonlinearly through the ballistic equations). L-BFGS-B is applied with 25 random initializations drawn uniformly from the feasible box; the solution with the lowest $\mathcal{L}$ is retained.

C Slingshot Kinematics and Trajectory Optimization

While holding the slingshot handle at a known pull position, the robot attempts to estimate the rubber band spring constant k in real time. The gravity-compensated joint efforts are mapped to an end-effector force via the Jacobian transpose, and this measured force is compared against the displacement from the cup equilibrium position to back-calculate k. If the new estimate is close to the current belief, the belief is updated; if the discrepancy is large, the estimate is discarded as unreliable.

Algorithm 8 Spring Constant Auto-Tuning

Require: Actual joint efforts $\boldsymbol{\tau}^a$, gravity torques $\boldsymbol{\tau}_{\text{grav}}$, current joint positions $\mathbf{q}^d$ (gripper excluded), cup equilibrium position $\mathbf{x}_{\text{eq}}$, handle length ℓ_h, current spring belief k, empirical correction c

Ensure: Updated spring constant estimate published to `/refine_launch`

$\boldsymbol{\tau}_{\text{net}} \leftarrow (\boldsymbol{\tau}^a - \boldsymbol{\tau}_{\text{grav}})_{\setminus g}$ ▷ Gravity-compensated effort, gripper excluded

$(\mathbf{x},\ J_v) \leftarrow \textsc{ForwardKin}(\mathbf{q}^d_{\setminus g})$

$\Delta\mathbf{x} \leftarrow \mathbf{x}_{\text{eq}} - \mathbf{x}$

$d \leftarrow \|\Delta\mathbf{x}\| - \ell_h$ ▷ Net spring displacement

$\mathbf{F}_{\text{meas}} \leftarrow J_v^{+\top}\, \boldsymbol{\tau}_{\text{net}}$ ▷ End-effector force via Jacobian transpose

if $d > 0.001\,\text{m}$ **then**

  $k_{\text{est}} \leftarrow c \cdot \|\mathbf{F}_{\text{meas}}\| \ / \ d$

else

  $k_{\text{est}} \leftarrow$ `None` ▷ Displacement too small to estimate reliably

end if

if $|k - k_{\text{est}}| < 5\,\text{N/m}$ **then**

  Publish k_{est}; flag `refine` ← `False` ▷ Converged — no further refinement needed

else if $|k - k_{\text{est}}| < 10\,\text{N/m}$ **then**

  Publish current belief k; flag `refine` ← `True` ▷ Moderate discrepancy — continue refining

else

  Discard estimate; flag `refine` ← `False` ▷ Large discrepancy — estimate unreliable

end if